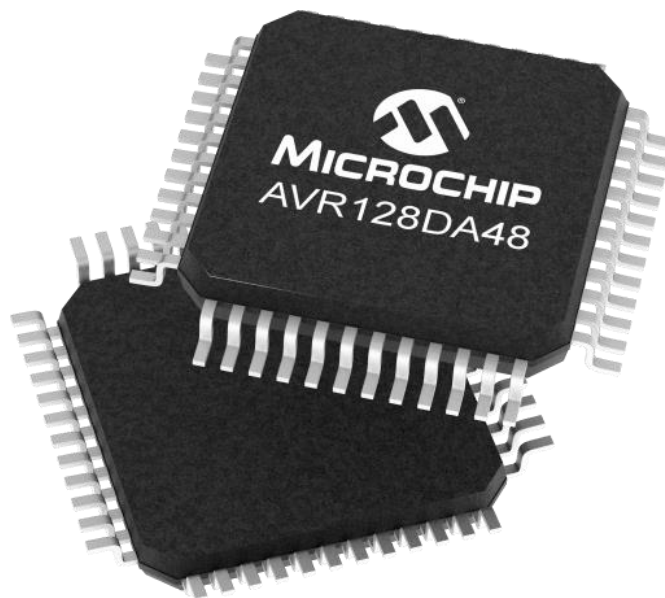
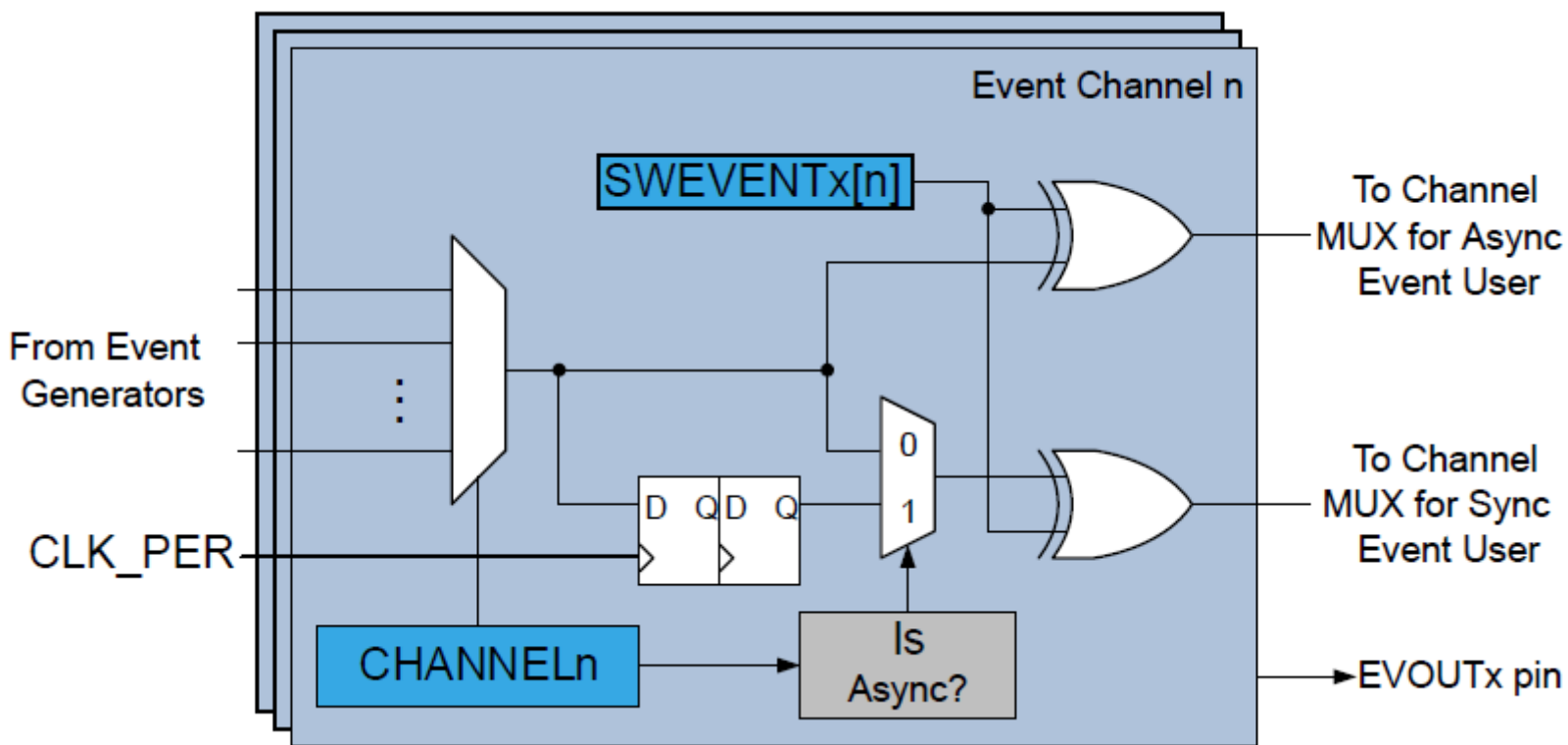


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 21 - Event System (EVSYS)



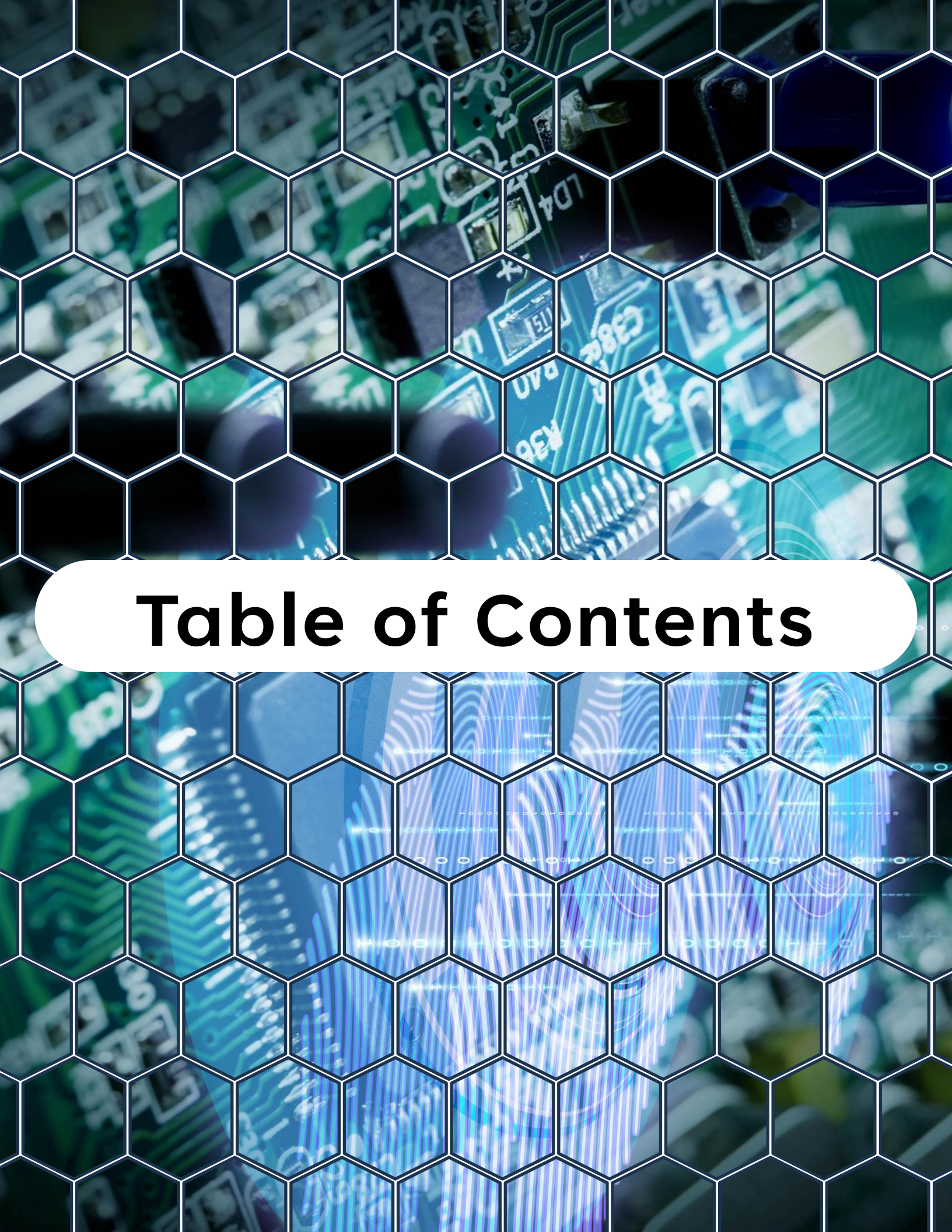
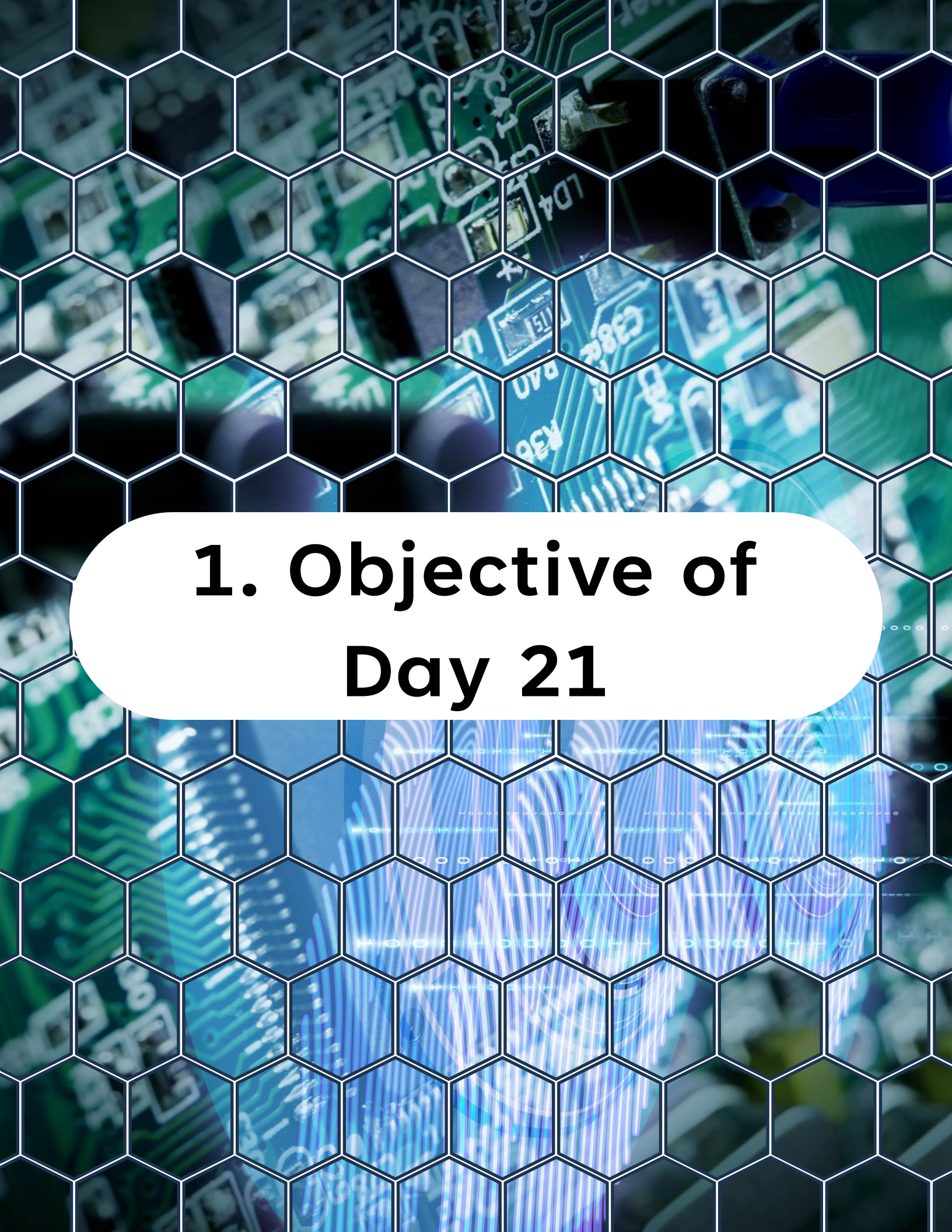


Table of Contents

Table of Contents

1. Objective of Day 21
2. What Is the Event System?
3. Event Channels
4. Why Use the Event System?
5. Architecture for Today's Lab
6. Step 1 - Configure TCB0 as Periodic Generator
7. Step 2 - Route TCB0 Overflow to EVSYS
8. Step 3 - Configure ADC for Event Trigger
9. Complete Lab - ADC Triggered by Timer (No CPU Trigger)
10. What You Should Observe
11. CPU Offloading Concept
12. What You Should Understand Now
13. Preview of Day 22



1. Objective of Day 21

1. Objective of Day 21

Today we study one of the most powerful architectural features of the AVR DA series:

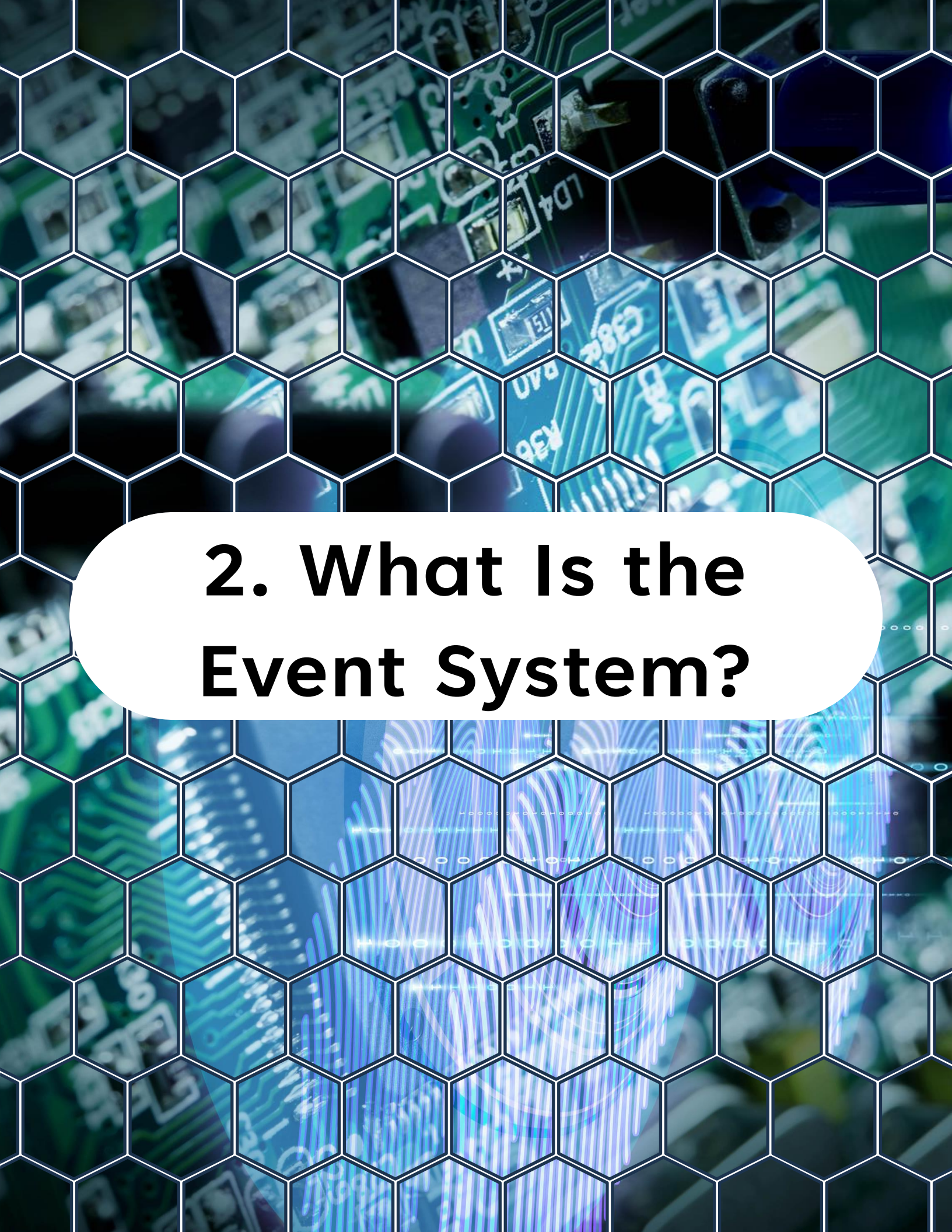
The **Event System (EVSYS)**.

The Event System allows peripherals to communicate directly with each other without CPU involvement.

By the end of today, you will understand:

- What event channels are
- How generators and users work
- How to build peripheral-to-peripheral pipelines
- How to offload the CPU
- How to trigger ADC conversions from a timer using hardware only

This is where embedded design becomes architectural.



2. What Is the Event System?

2. What Is the Event System?

The Event System is a hardware routing network inside the MCU.

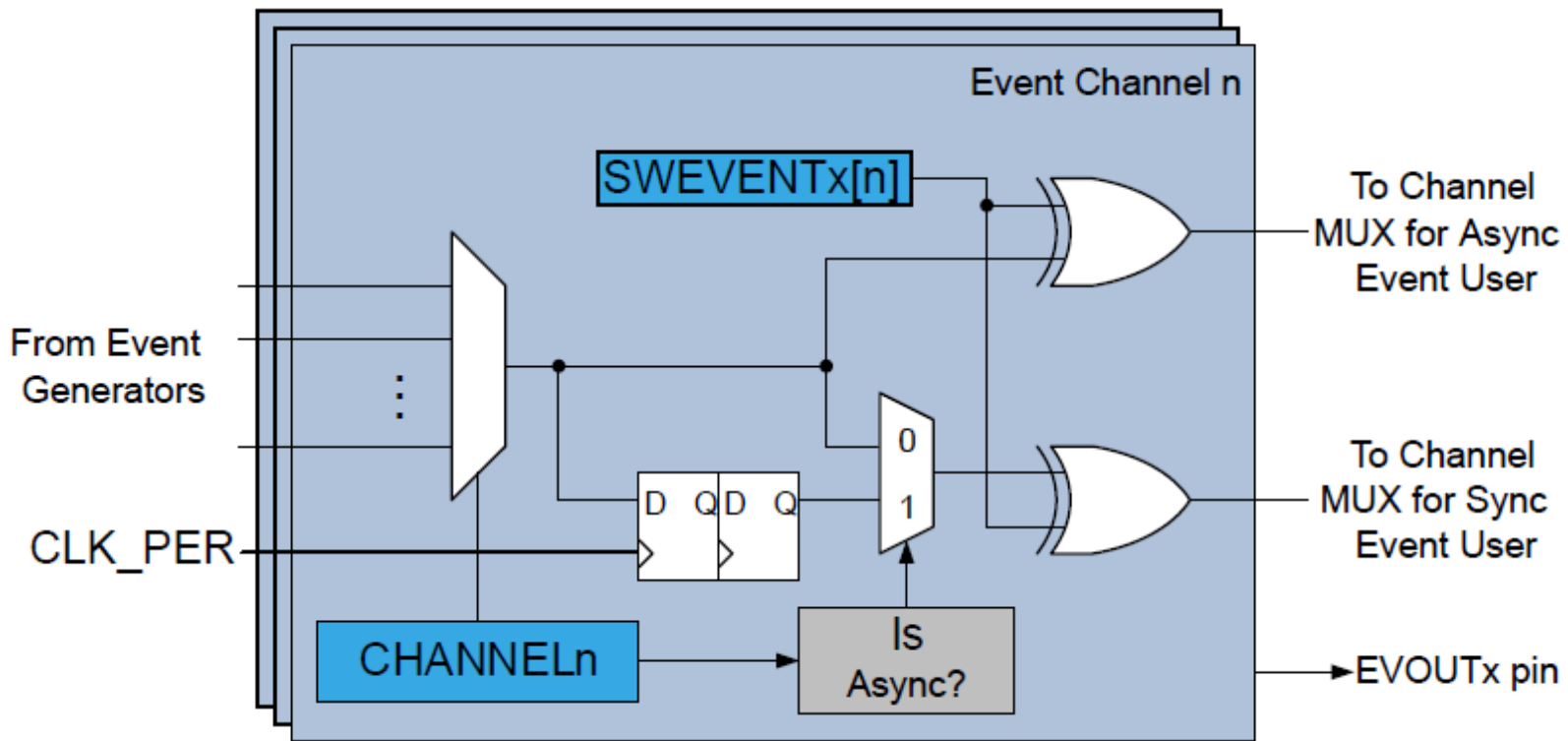
It allows:

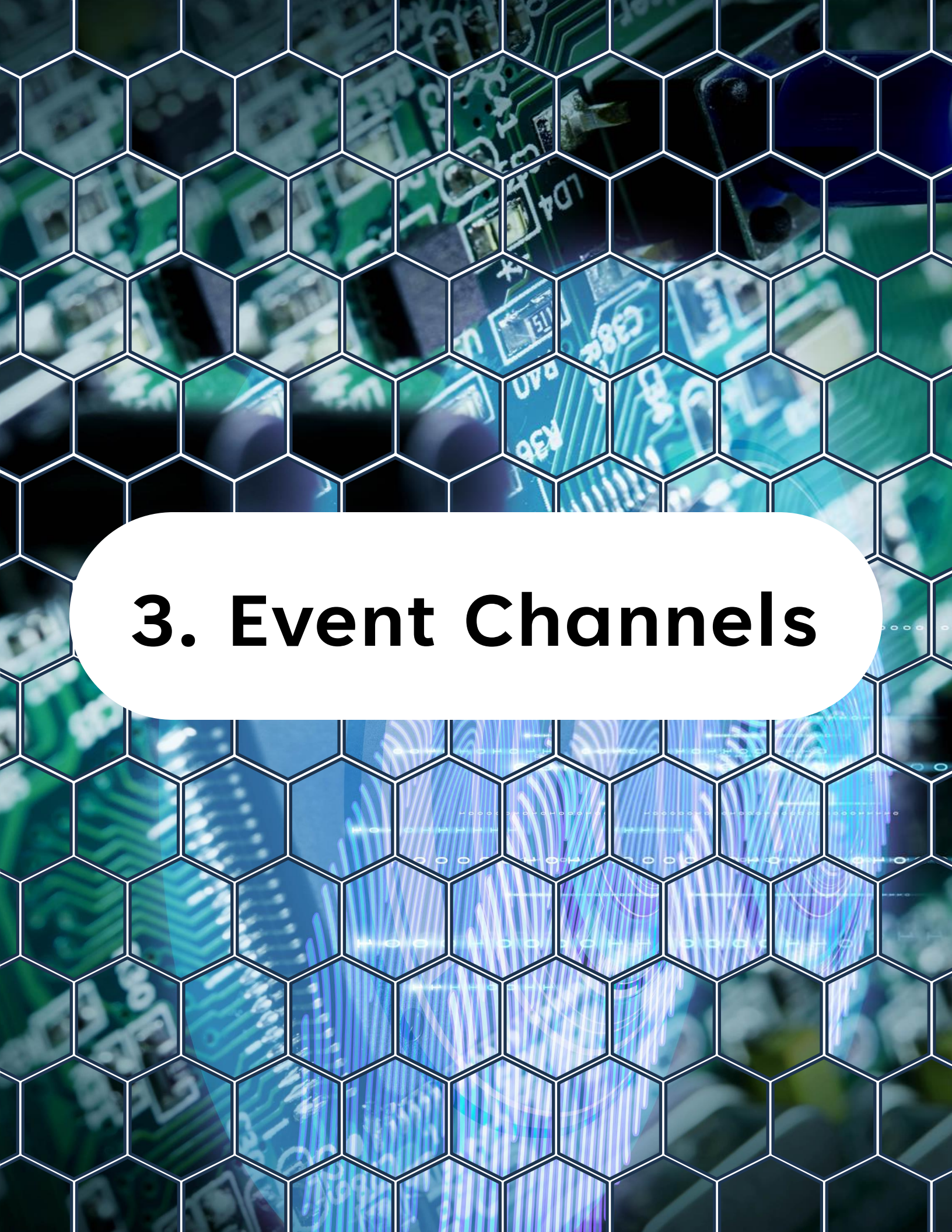
- Peripheral A -> Event Channel -> Peripheral B
- No interrupt.
- No firmware intervention.
- No polling.

Events propagate in hardware, typically within one clock cycle.

2. What Is the Event System?

Event System Block Diagram





3. Event Channels

3. Event Channels

An Event Channel is a hardware "wire" inside the MCU.

Each channel has:

- One generator
- One or more users

Think of it as:

Generator -> Channel -> User(s)

Example:

TCB0 overflow -> **EVSYS** Channel 0 -> **ADC** start conversion

3. Event Channels

Generator

A generator is a peripheral that produces an event.

Examples:

- Timer overflow
- Comparator output change
- External pin edge
- RTC compare

You select it by writing to:

EVSYS.CHANNELn

3. Event Channels

User

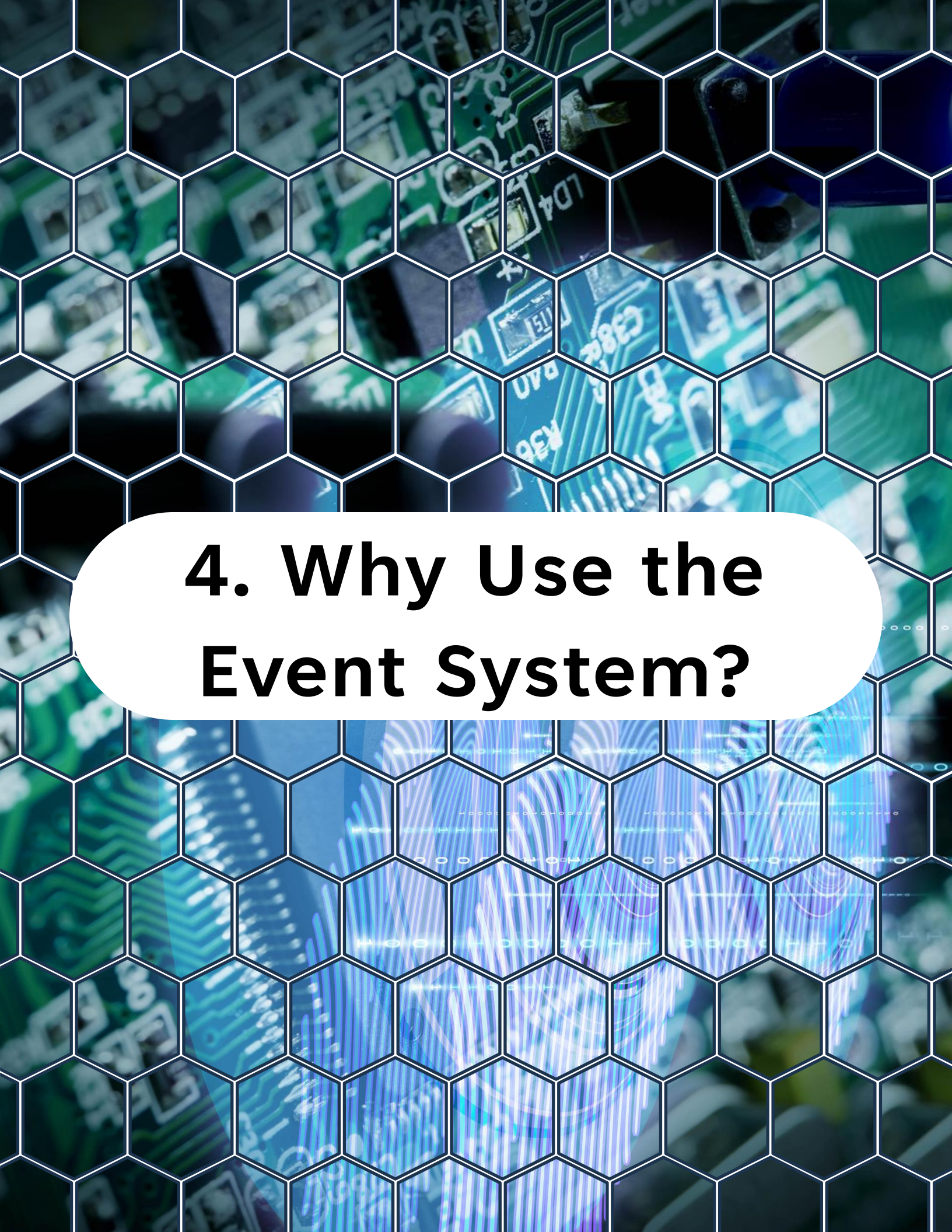
A user is a peripheral that reacts to the event.

Examples:

- ADC start conversion
- Timer capture
- PWM fault
- Counter reset

You connect it using:

`EVSYS.USER_xxx`



4. Why Use the Event System?

4. Why Use the Event System?

Without EVSYS:

Timer interrupt -> ISR -> Start ADC -> Wait ->
Read result

Problems:

- Interrupt latency
- Firmware overhead
- Jitter
- Increased complexity

4. Why Use the Event System?

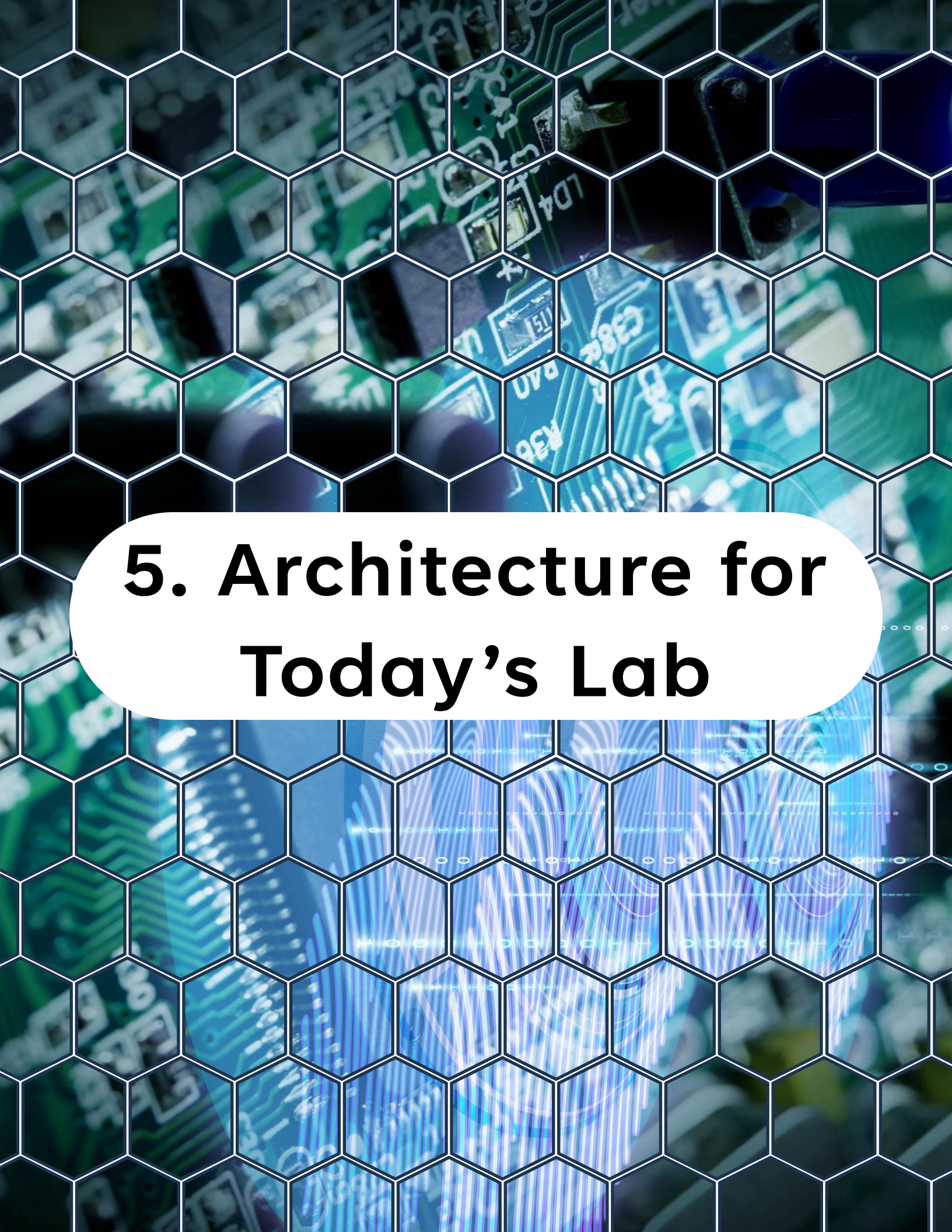
With EVSYS:

Timer -> ADC start

Hardware handles it.

Benefits:

- Deterministic timing
- Zero CPU overhead
- Reduced power consumption
- Cleaner architecture



5. Architecture for Today's Lab

5. Architecture for Today's Lab

We will build:

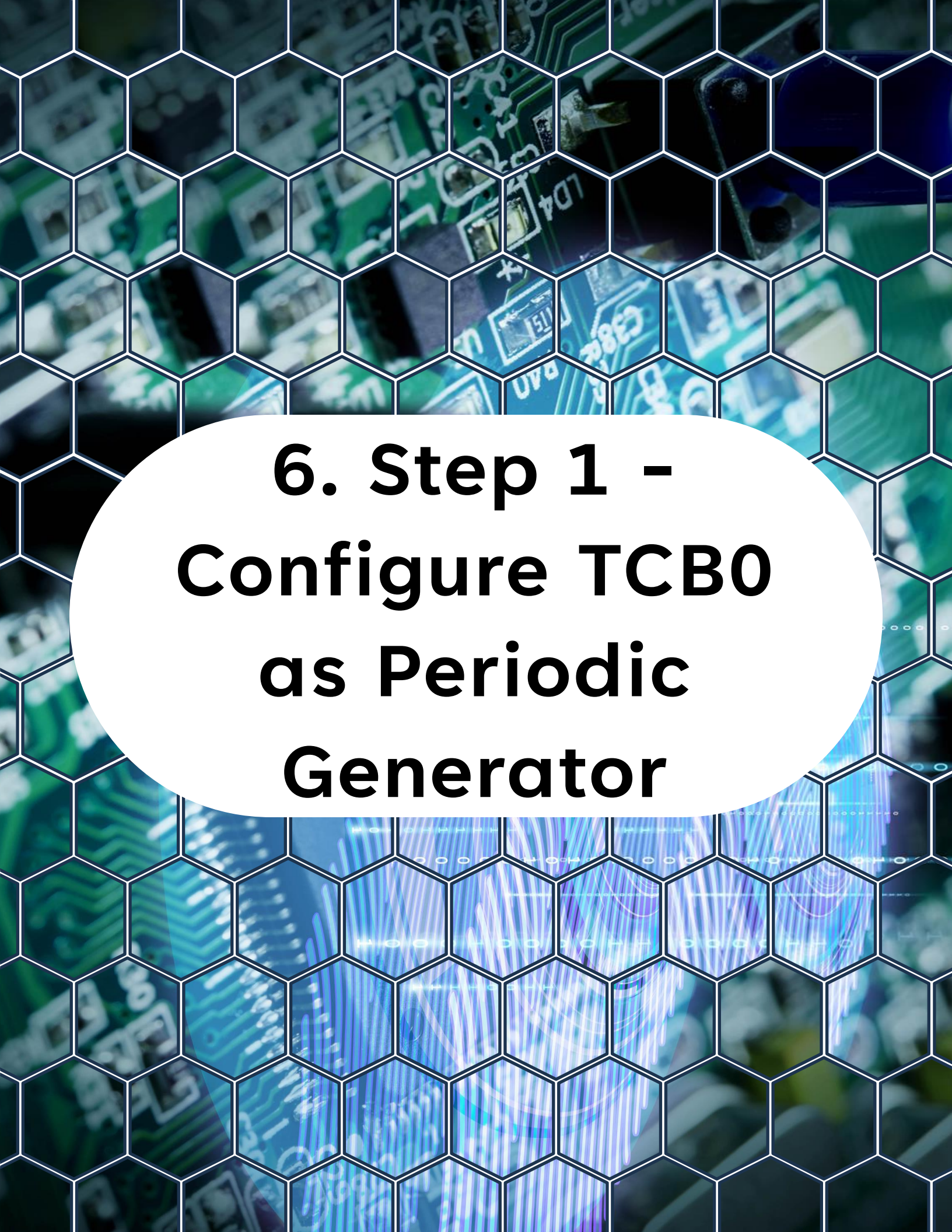
TCB0 periodic event-> **EVSYS** Channel 0-> **ADC0**
start conversion

ADC conversion will happen at a fixed rate
without calling:

```
ADC0.COMMAND = ADC_STCONV_bm;
```

The CPU will only read the results.

In this scenario, the TCB0 is the **Generator**, and
the ADC0 is the **User** that consumes the event.



6. Step 1 - Configure TCB0 as Periodic Generator

6. Step 1 - Configure TCB0 as Periodic Generator

We configure **TCB0** to generate a periodic overflow event.

Example:

- $F_{\text{CPU}} = 24 \text{ MHz}$
- $\text{CLK_PER} = F_{\text{CPU}} / 2 = 12 \text{ MHz}$
- Target sample rate = 1 kHz

$$12,000,000 / 1000 = 12000$$

Code Example



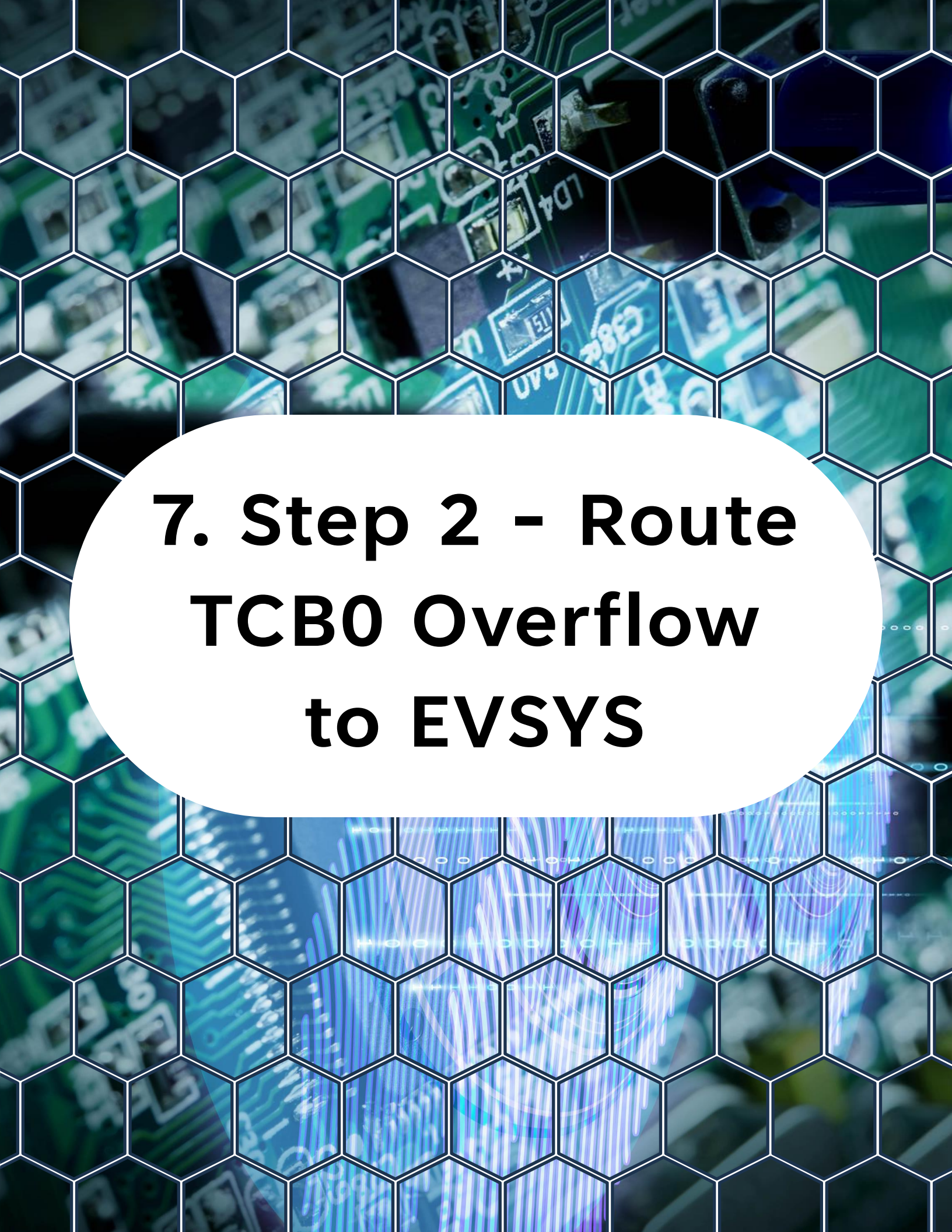
```
1  /**
2   * @brief Initialize TCB0 to generate periodic overflow at 1 kHz.
3   */
4  static void TCB0_init(void)
5  {
6      // Disable TCB0 before configuration
7      TCB0.CTRLA &= ~(1 << TCB_ENABLE_bp);
8
9      // Periodic interrupt mode
10     TCB0.CTRLB = TCB_CNTMODE_INT_gc;
11
12     // Set TCB0 counter value to 12000 (12 MHz / 1 kHz)
```

6. Step 1 - Configure TCB0 as Periodic Generator

Code Example

```
1  /**
2   * @brief Initialize TCB0 to generate periodic overflow at 1 kHz.
3   */
4  static void TCB0_init(void)
5  {
6      // Disable TCB0 before configuration
7      TCB0.CTRLA &= ~(1 << TCB_ENABLE_bp);
8
9      // Periodic interrupt mode
10     TCB0.CTRLB = TCB_CNTMODE_INT_gc;
11
12     // Set TCB0 to periodic interrupt mode with a period of 1 ms (1 kHz)
13     // F_CPU = 24 MHz
14     // CLK_PER = F_CPU / TCB0_Prescaler = 12 MHz
15     // TCB0 Period = (CCMP + 1) / CLK_PER
16     // CCMP = (TCB0 Period * CLK_PER) - 1 = (0.001 s * 12,000,000 Hz) - 1
17     // CCMP = 12000 - 1 = 11999
18     TCB0.CCMP = 11999;
19
20     // Enable timer
21     TCB0.CTRLA = TCB_CLKSEL_DIV2_gc | TCB_ENABLE_bm;
22 }
```

TCB0 now overflows every 1 ms.



7. Step 2 - Route TCB0 Overflow to EVSYS

7. Step 2 - Route TCB0 Overflow to EVSYS

We configure:

- Generator = **TCB0** overflow
- **Channel** = 0
- We configure **ADC** to use event input as conversion trigger.

```
1  /**
2   * @brief Configure EVSYS Channel 0 to use TCB0 overflow.
3   */
4  static void EVSYS_init(void)
5  {
6      // Configure EVSYS Channel 0 to use TCB0 overflow as
7      // the event source
8      EVSYS.CHANNEL0 = EVSYS_CHANNEL0_TCB0_OVF_gc;
9
10     // Configure ADC0 to use event channel 0 as the
11     // start trigger
12     EVSYS.USERADC0START = EVSYS_USER_CHANNEL0_gc;
13 }
```

Now Channel 0 carries TCB0 overflow events, and timer overflow triggers ADC conversion.

**8. Step 3 -
Configure ADC
for Event Trigger**

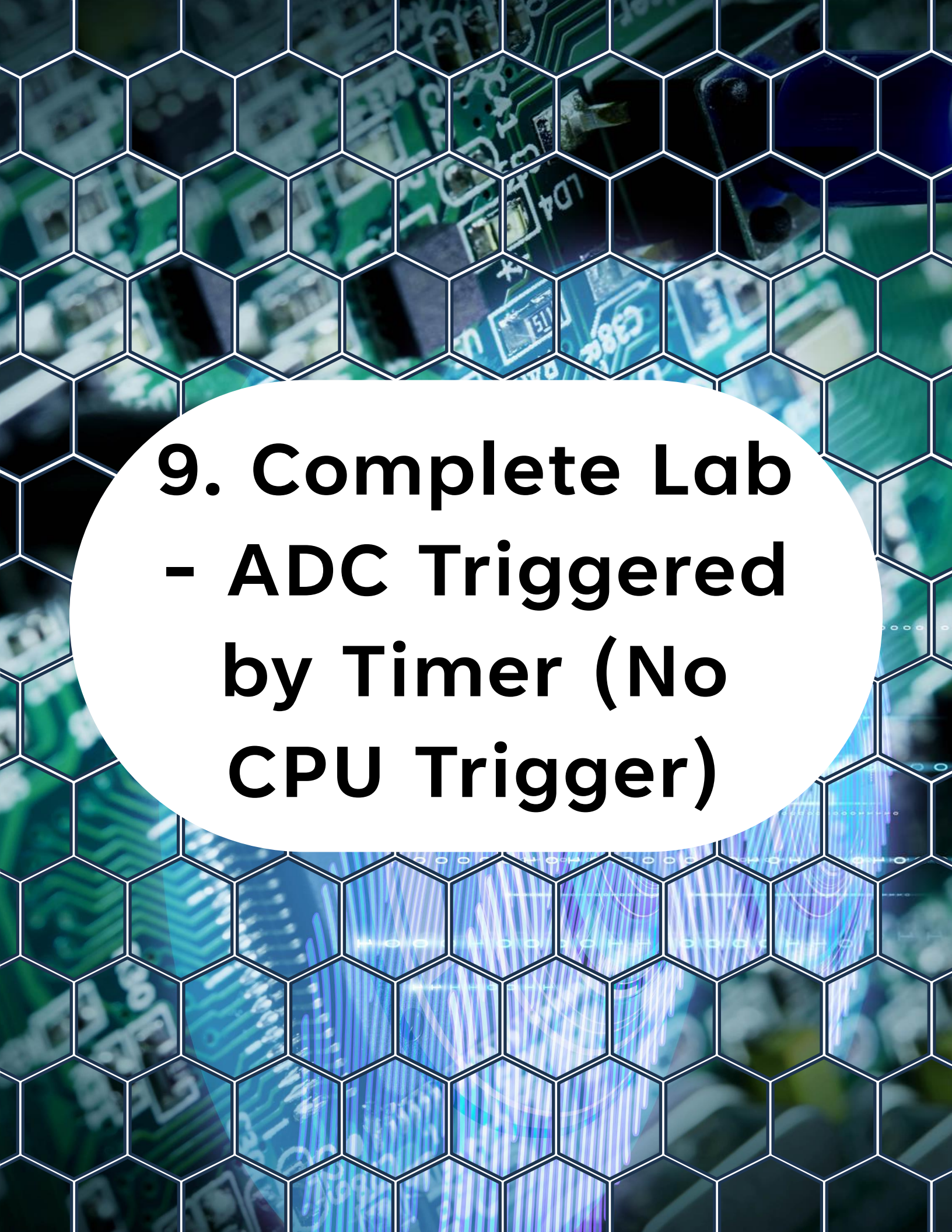
8. Step 3 - Configure ADC for Event Trigger

We enable:

- Event input mode
- Result ready interrupt (optional)

```
1  /**
2   * @brief Initialize ADC0 for event-triggered sampling.
3   */
4  static void ADC0_init(void)
5  {
6      // Set 2.048 V reference for ADC0 in the VREF peripheral
7      VREF.ADC0REF = VREF_REFSEL_2V048_gc;
8
9      // Select input channel AIN0 for ADC0
10     ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
11
12     // Set ADC0 clock prescaler to divide by 16
13     ADC0.CTRLA = ADC_PRESC_DIV16_gc;
14
15     // Enables the event input as trigger for starting
16     // an ADC conversion
17     ADC0.EVCTRL = ADC_STARTEI_bm;
18
19     // Enable ADC0 with 12-bit resolution
20     ADC0.CTRLA = ADC_ENABLE_bm
21                 | ADC_RESSEL_12BIT_gc;
22 }
```

Now ADC waits for hardware event.



9. Complete Lab - ADC Triggered by Timer (No CPU Trigger)

9. Complete Lab - ADC Triggered by Timer (No CPU Trigger)

```
1  /**
2   * @file main.c
3   * @brief ADC triggered by TCB0 via Event System.
4   */
5  #include <avr/io.h>
6  #include <avr/interrupt.h>
7  #include <stdint.h>
8
9  // Global variable to store the latest ADC value
10 volatile uint16_t g_adc_value;
11
12 /**
13  * @brief Initialize TCB0 to generate periodic overflow at 1 kHz.
14  */
15 static void TCB0_init(void)
16 {
17     // Disable TCB0 before configuration
18     TCB0.CTRLA &= ~(1 << TCB_ENABLE_bp);
19
20     // Periodic interrupt mode
21     TCB0.CTRLB = TCB_CNTMODE_INT_gc;
22
23     // Set TCB0 to periodic interrupt mode with a period of 1 ms (1 kHz)
24     // F_CPU = 24 MHz
25     // CLK_PER = F_CPU / TCB0_Prescaler = 12 MHz
26     // TCB0 Period = (CCMP + 1) / CLK_PER
27     // CCMP = (TCB0 Period * CLK_PER) - 1 = (0.001 s * 12,000,000 Hz) - 1
28     // CCMP = 12000 - 1 = 11999
29     TCB0.CCMP = 11999;
30
31     // Enable timer
32     TCB0.CTRLA = TCB_CLKSEL_DIV2_gc | TCB_ENABLE_bm;
33 }
34
35 /**
36  * @brief Configure EVSYS Channel 0 to use TCB0 overflow.
37  */
```

9. Complete Lab - ADC Triggered by Timer (No CPU Trigger)

```
35 /**
36  * @brief Configure EVSYS Channel 0 to use TCB0 overflow.
37  */
38 static void EVSYS_init(void)
39 {
40     // Configure EVSYS Channel 0 to use TCB0 overflow as
41     // the event source
42     EVSYS.CHANNEL0 = EVSYS_CHANNEL0_TCB0_OVF_gc;
43
44     // Configure ADC0 to use event channel 0 as the
45     // start trigger
46     EVSYS.USERADC0START = EVSYS_USER_CHANNEL0_gc;
47 }
48
49 /**
50  * @brief Initialize ADC0 for event-triggered sampling.
51  */
52 static void ADC0_init(void)
53 {
54     // Set 2.048 V reference for ADC0 in the VREF peripheral
55     VREF.ADC0REF = VREF_REFSEL_2V048_gc;
56
57     // Select input channel AIN0 for ADC0
58     ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
59
60     // Set ADC0 clock prescaler to divide by 16
61     ADC0.CTRLA = ADC_PRESC_DIV16_gc;
62
63     // Enable ADC0 result ready interrupt
64     ADC0.INTCTRL = ADC_RESRDY_bm;
65
66     // Enables the event input as trigger for starting
67     // an ADC conversion
68     ADC0.EVCTRL = ADC_STARTEI_bm;
69
70     // Enable ADC0 with 12-bit resolution
71     ADC0.CTRLA = ADC_ENABLE_bm
72                 | ADC_RESSEL_12BIT_gc;
73 }
```

9. Complete Lab - ADC Triggered by Timer (No CPU Trigger)

```
74
75 /**
76  * @brief ADC0 Result-Ready ISR
77  */
78 ISR(ADC0_RESRDY_vect)
79 {
80     ADC0.INTFLAGS = ADC_RESRDY_bm;
81     g_adc_value = ADC0.RES;
82 }
83
84 /**
85  * @brief Application entry point
86  */
87 int main(void)
88 {
89     // Initialize TCB0 to generate overflow events at 1 kHz
90     TCB0_init();
91
92     // Configure EVSYS to route TCB0 overflow events to ADC0 start trigger
93     EVSYS_init();
94
95     // Initialize ADC0. ADC0 will start sampling automatically at 1 kHz
96     ADC0_init();
97
98     // Enable global interrupts
99     sei();
100
101     while (1)
102     {
103         /* ADC sampled at exactly 1 kHz by hardware */
104     }
105 }
```

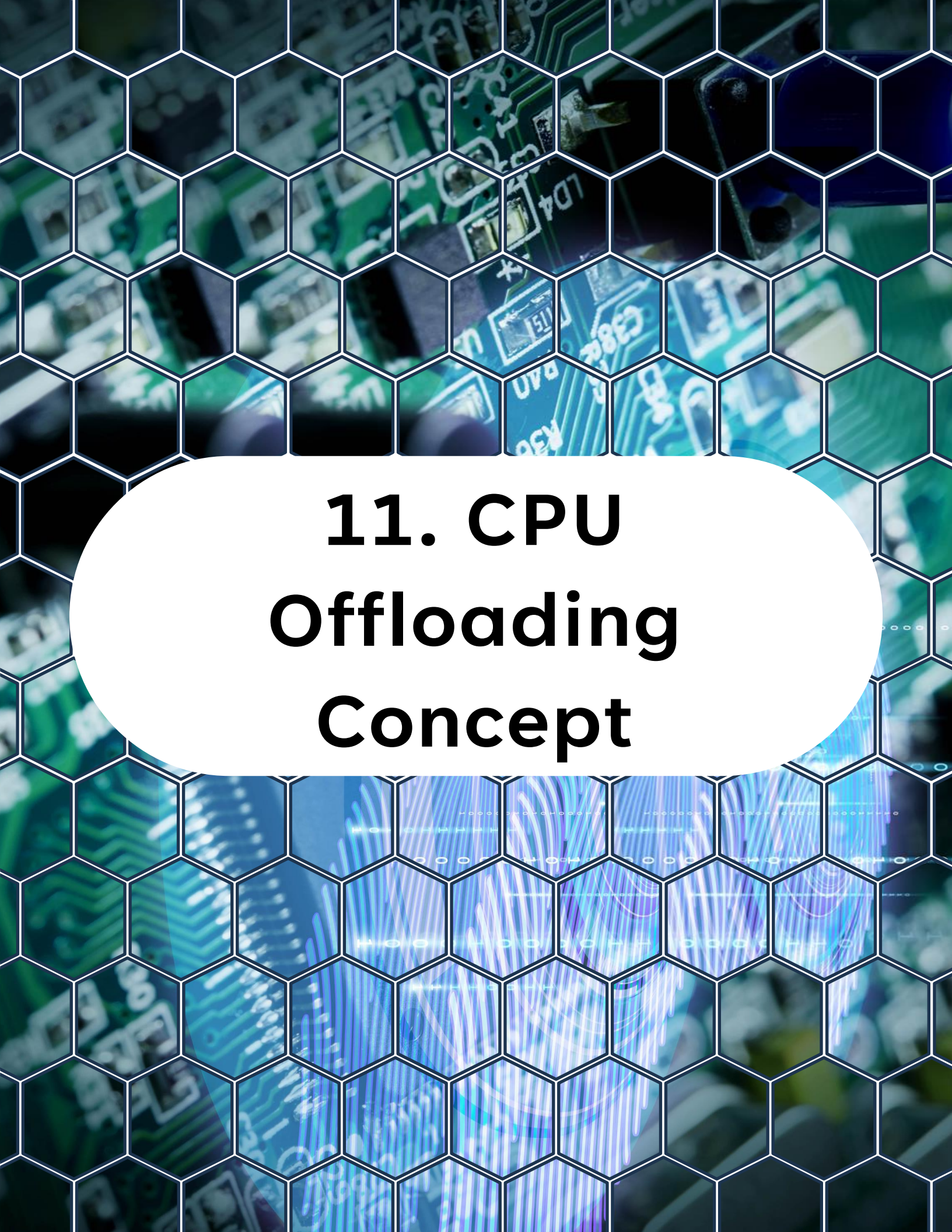


10. What You Should Observe

10. What You Should Observe

- ADC samples exactly at 1 kHz
- No ADC start command in firmware
- CPU free for other tasks
- Sampling interval deterministic

Measure sampling rate by toggling a GPIO in ISR if needed.



11. CPU Offloading Concept

11. CPU Offloading Concept

With **EVSYS** you can build chains like:

**Timer -> ADC -> Comparator -> PWM
shutdown**

All without CPU involvement.

This reduces:

- Interrupt load
- Power consumption
- Firmware complexity

And improves:

- Timing precision
- Determinism
- Reliability



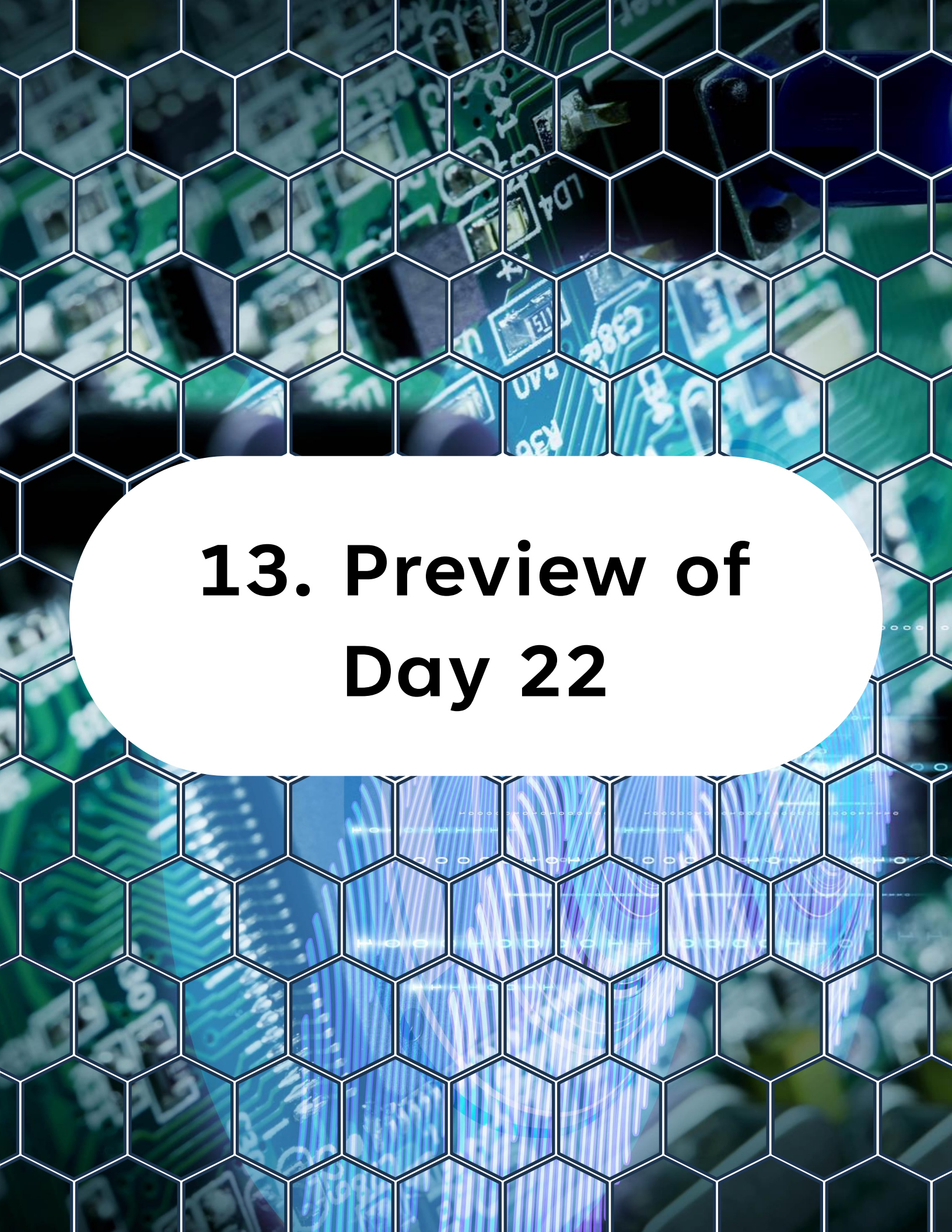
12. What You Should Understand Now

12. What You Should Understand Now

You now know:

1. What event channels are
2. How generators and users work
3. How to build peripheral pipelines
4. How to offload the CPU
5. How to trigger ADC from a timer

This is hardware-driven embedded architecture.



13. Preview of Day 22

13. Preview of Day 22

Next:

Sleep Modes and Power Management

We will combine:

- Event-driven ADC
- Low-power sleep
- Wake-up sources

To build an efficient embedded system that samples without wasting energy.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>